

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

CSE 4802: Software Maintenance Lab

Lab Project
Software Maintenance Activities

Antara Arifa Mullick
Student ID: 210042132
Program: B.Sc. in SWE
Department of CSE

Task Author: Lutfun Nahar Lota
CSE, IUT

Contents

1 Selected Project **1**

2 Overall Maintenance Summary **1**

3 Maintenance Works Performed on the Project Using Lab Tasks **2**

3.1 Program Comprehension using Static and Dynamic Analysis – Lab 1 2

3.2 Structural Reverse Engineering and Impact Analysis – Lab 2 . 7

3.3 Runtime Comprehension and Performance Analysis – Lab 3 . 11

3.4 Clone Detection and Refactoring Opportunity Analysis – Lab 5 13

4 Corrective Maintenance **14**

4.1 Problem Identified 14

4.2 Program Comprehension 15

4.3 Change Management 15

4.4 Impact Analysis 15

4.5 Reverse Engineering 15

4.6 Refactoring 16

4.7 Result 16

5 Adaptive Maintenance **17**

5.1 Maintenance Requirement . . . 17

5.2 Program Comprehension 17

5.3 Change Management 17

5.4 Impact Analysis 18

5.5 Reverse Engineering 18

5.6 Refactoring 19

5.7 Result 19

6 Preventive Maintenance **20**

6.1 Maintenance Risk 20

6.2 Program Comprehension 20

6.3 Change Management 21

6.4 Impact Analysis 21

6.5 Reverse Engineering 21

6.6 Refactoring 22

6.7 Result 22

7 Perfective Maintenance **22**

7.1 Improvement Requirement . . . 22

7.2 Program Comprehension 23

7.3 Change Management 23

7.4 Impact Analysis 23

7.5 Reverse Engineering 24

7.6 Refactoring 24

7.7 Result 24

8 Conclusion **25**

1 Selected Project

The selected project for the maintenance lab project was **Heart Disease Prediction**. It is a Python-based machine learning application that uses a heart disease dataset and a Logistic Regression model to predict whether a sample belongs to the heart-disease class.

The main project workflow is:

1. Load the dataset from `heart_disease_data.csv`.
2. Separate the feature columns and the target column.
3. Split the data into training and testing sets.
4. Train a Logistic Regression model.
5. Evaluate training and testing accuracy.
6. Validate a sample input and generate a prediction.
7. Convert the prediction to a readable result.

The project was suitable for software maintenance activities because it contains clear functions for data loading, preparation, model training, evaluation, validation, and prediction. The same project was therefore used for program comprehension, structural analysis, runtime analysis, clone detection, and the four categories of software maintenance.

The original model produced a training accuracy of approximately **0.8554** and a testing accuracy of approximately **0.8033**. These values remained the expected baseline while the maintenance changes were performed.

2 Overall Maintenance Summary

Category	Main Change	Maintenance Activities Demonstrated
Corrective	Rejected NaN and infinity sample values	Program comprehension, documented change management, reference-based impact analysis, AST reverse engineering, behavioral refactoring, and verification.
Adaptive	Made resource paths independent of the current working directory	Path-focused program comprehension, change planning, path impact analysis, AST reverse engineering, centralized path refactoring, and execution from another directory.

Category	Main Change	Maintenance Activities Demonstrated
Preventive	Added dataset schema validation	Schema comprehension, preventive change planning, dependency impact analysis, AST reverse engineering, constant/schema refactoring, and early failure for invalid data.
Perfective	Added custom sample input and predicted class probability	Prediction-flow comprehension, improvement planning, impact analysis, AST reverse engineering, reusable prediction/input refactoring, and enhanced output.

The earlier lab tasks supported these activities as follows:

Lab Task	**Maintenance Contribution**
Lab 1	Static/dynamic program comprehension, metrics, runtime tracing, profiling, and quality analysis.
Lab 2	Structural program comprehension, CPG-based reverse engineering, dependency and impact analysis, and Neo4j visualization.
Lab 3	Runtime comprehension, debugging support, execution tracing, and performance profiling using Loguru, PySnooper, VizTracer, and SnakeViz.
Lab 5	Clone detection, duplication analysis, preventive-maintenance insight, and identification of refactoring opportunities.

Lab 4 was performed as a separate reverse-engineering exercise because its compiled/native and Java reverse-engineering activities were not naturally integrated into this Python machine-learning project. Reverse-engineering evidence for the integrated project was instead provided by the Lab 2 CPG work and the focused AST analyses used during the four maintenance categories.

3 Maintenance Works Performed on the Project Using Lab Tasks

3.1 Program Comprehension using Static and Dynamic Analysis – Lab 1

Lab 1 was mainly used for **program comprehension**. Static analysis was performed using Python AST processing and source metrics. At that stage, the analysis reported **926 AST nodes** and **9 functions**. Function-level metrics such as calls, conditions, returns, and approximate cyclomatic complexity were also generated. This helped identify important

functions and more complex decision-oriented parts such as prediction conversion, model-status checking, and sample validation.

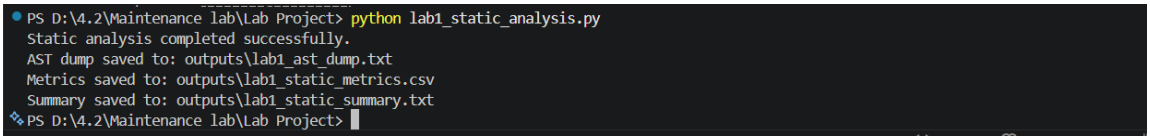


Figure 1: Static analysis execution and generated analysis outputs for the Heart Disease Prediction project.

Dynamic analysis was then used to understand what actually executed at runtime. The dynamic summary recorded **75 traced events** and **61 unique executed lines**, including calls to dataset loading, data preparation, model training, evaluation, and the main workflow. This complemented the static structure with observed execution behavior.

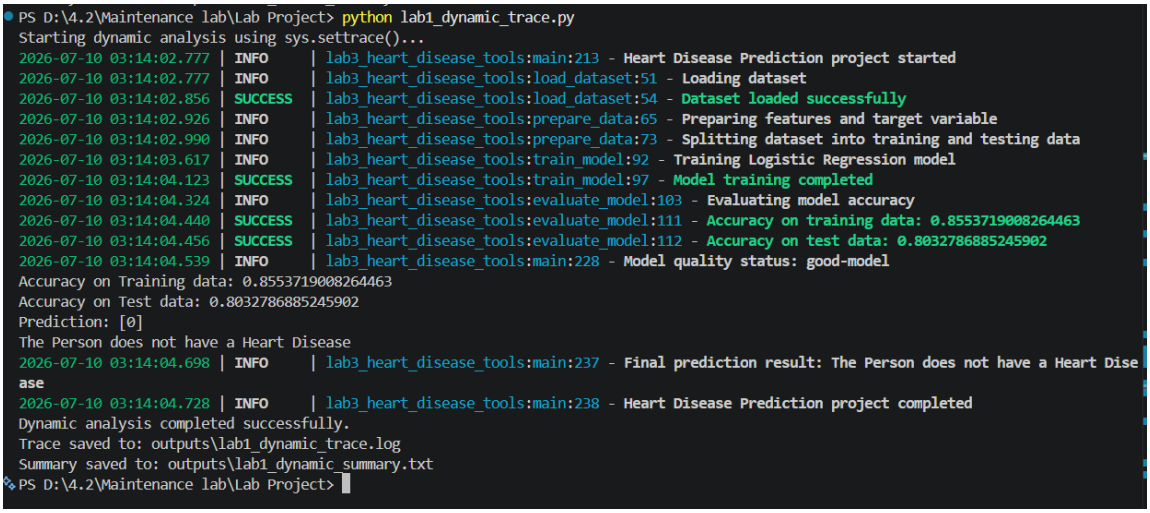
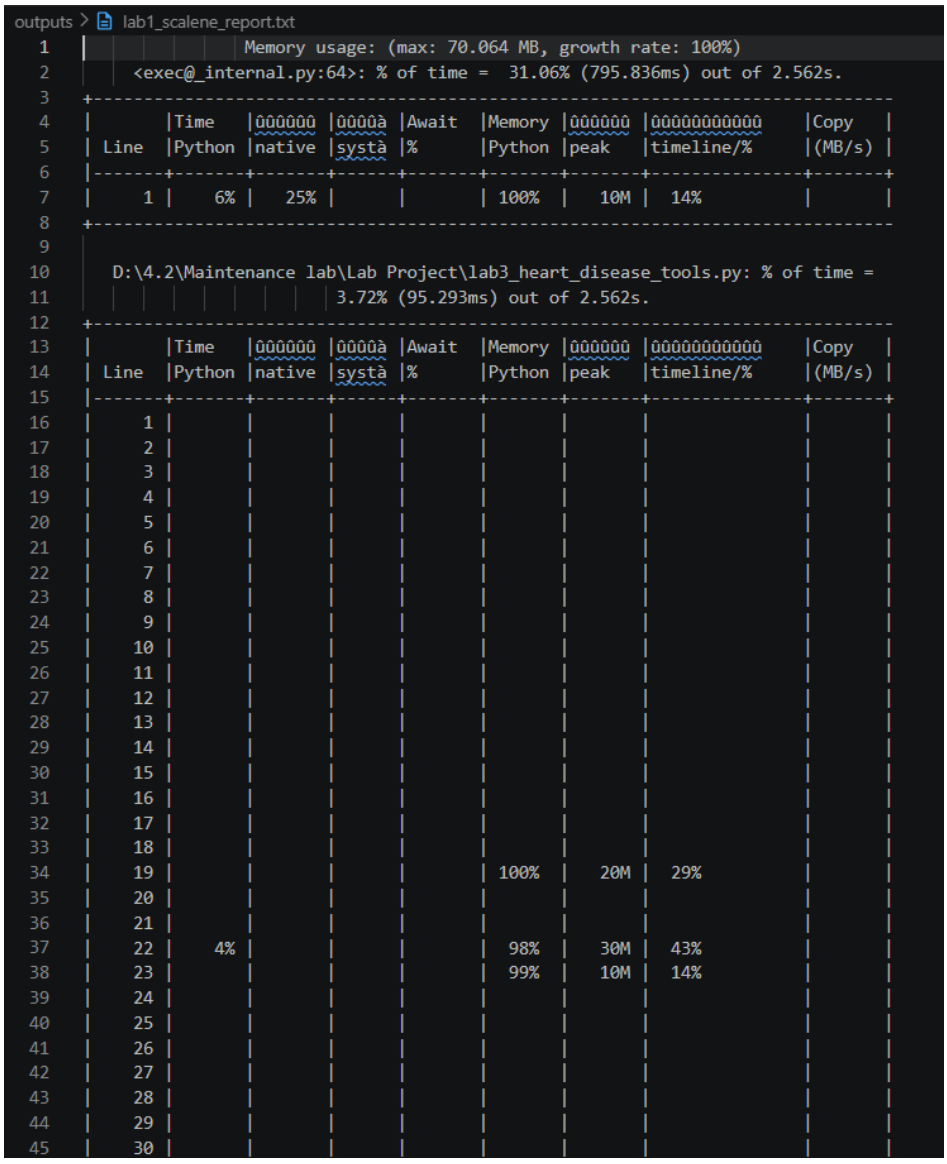


Figure 2: Dynamic execution trace of the project showing the runtime workflow and generated trace output.

Performance-oriented comprehension was also supported by Scalene profiling. The profiler helped identify where execution time and resource use occurred during project execution.



outputs > lab1_scalene_report.txt

```
                Memory usage: (max: 70.064 MB, growth rate: 100%)
    <exec@_internal.py:64>: % of time =  31.06% (795.836ms) out of 2.562s.
+-------------------------------------------------------------------------------
|       |Time   |ûûûûûû |ûûûûà |Await  |Memory |ûûûûûû |ûûûûûûûûûûûû    |Copy   |
| Line  |Python |native |systà |%      |Python |peak   |timeline/%     |(MB/s) |
|-------+-------+-------+------+-------+-------+-------+---------------+-------+
|     1 |    6% |   25% |      |       | 100%  |   10M |  14%          |       |
+-------------------------------------------------------------------------------

   D:\4.2\Maintenance lab\Lab Project\lab3_heart_disease_tools.py: % of time =
                3.72% (95.293ms) out of 2.562s.
+-------------------------------------------------------------------------------
|       |Time   |ûûûûûû |ûûûûà |Await  |Memory |ûûûûûû |ûûûûûûûûûûûû    |Copy   |
| Line  |Python |native |systà |%      |Python |peak   |timeline/%     |(MB/s) |
|-------+-------+-------+------+-------+-------+-------+---------------+-------+
|     1 |       |       |      |       |       |       |               |       |
|     2 |       |       |      |       |       |       |               |       |
|     3 |       |       |      |       |       |       |               |       |
|     4 |       |       |      |       |       |       |               |       |
|     5 |       |       |      |       |       |       |               |       |
|     6 |       |       |      |       |       |       |               |       |
|     7 |       |       |      |       |       |       |               |       |
|     8 |       |       |      |       |       |       |               |       |
|     9 |       |       |      |       |       |       |               |       |
|    10 |       |       |      |       |       |       |               |       |
|    11 |       |       |      |       |       |       |               |       |
|    12 |       |       |      |       |       |       |               |       |
|    13 |       |       |      |       |       |       |               |       |
|    14 |       |       |      |       |       |       |               |       |
|    15 |       |       |      |       |       |       |               |       |
|    16 |       |       |      |       |       |       |               |       |
|    17 |       |       |      |       |       |       |               |       |
|    18 |       |       |      |       |       |       |               |       |
|    19 |       |       |      |       | 100%  |   20M |  29%          |       |
|    20 |       |       |      |       |       |       |               |       |
|    21 |       |       |      |       |       |       |               |       |
|    22 |    4% |       |      |       |  98%  |   30M |  43%          |       |
|    23 |       |       |      |       |  99%  |   10M |  14%          |       |
|    24 |       |       |      |       |       |       |               |       |
|    25 |       |       |      |       |       |       |               |       |
|    26 |       |       |      |       |       |       |               |       |
|    27 |       |       |      |       |       |       |               |       |
|    28 |       |       |      |       |       |       |               |       |
|    29 |       |       |      |       |       |       |               |       |
|    30 |       |       |      |       |       |       |               |       |
```

Figure 3: Scalene profiling output used to understand runtime performance characteristics.

SonarCloud analysis provided an additional maintainability and code-quality view of the project. The summary showed no open security or reliability issues and four maintainability issues, while the detailed issue list identified low-severity code smells that could be considered during later cleanup.

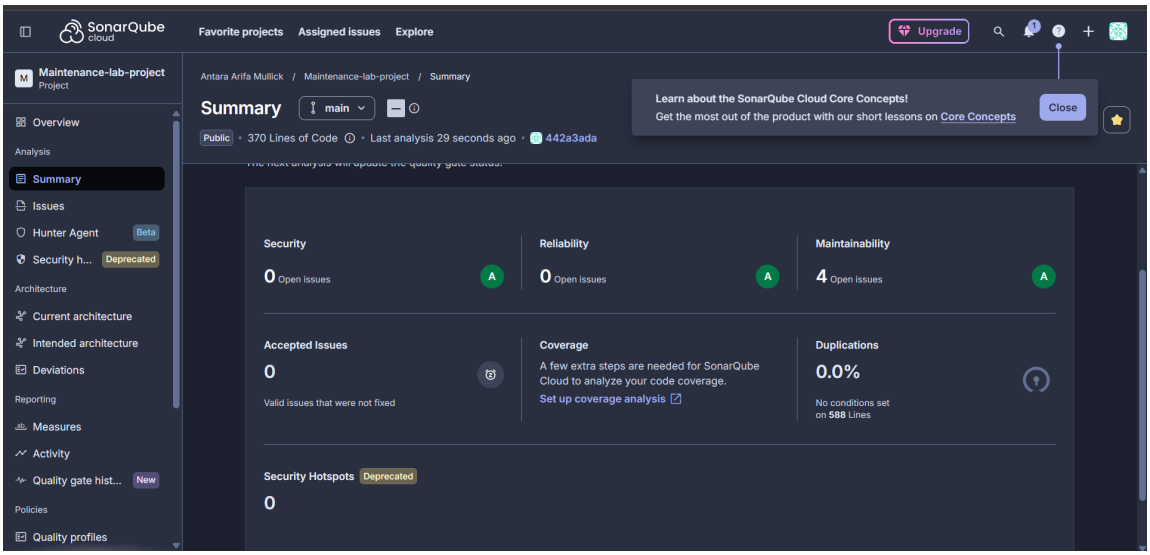


Figure 4: SonarCloud project summary used as an additional code-quality and maintainability view.

The issue view showed naming-convention findings for the local variables `X_train_prediction` and `X_test_prediction`. These findings illustrate how static quality analysis can identify consistency and readability concerns even when the program is functionally correct.

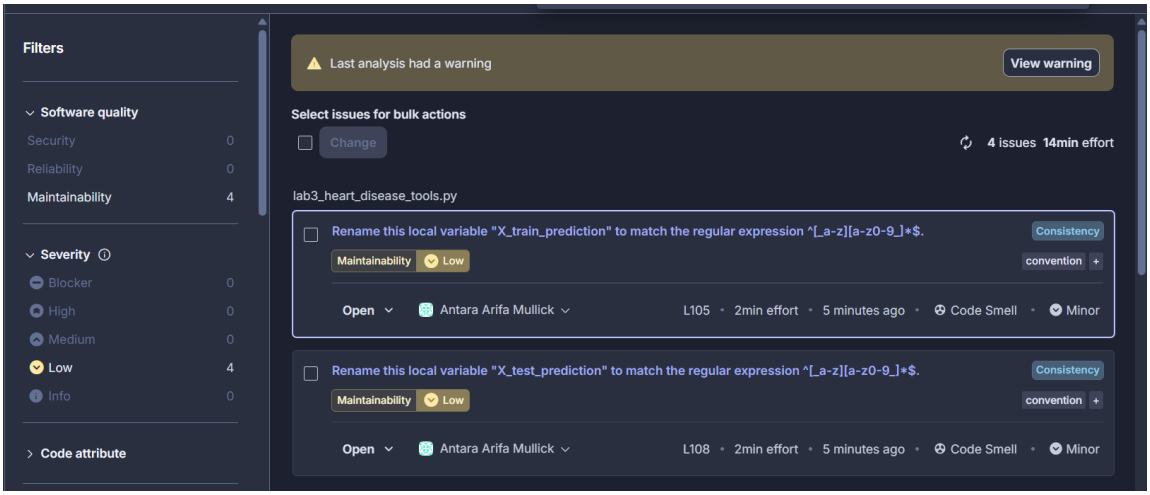


Figure 5: SonarCloud maintainability issues highlighting local-variable naming conventions.

The remaining issue view identified unused local test variables such as `X_test` and `Y_test`. Such findings are useful during maintenance because unused variables increase clutter and can make test intent harder to understand.

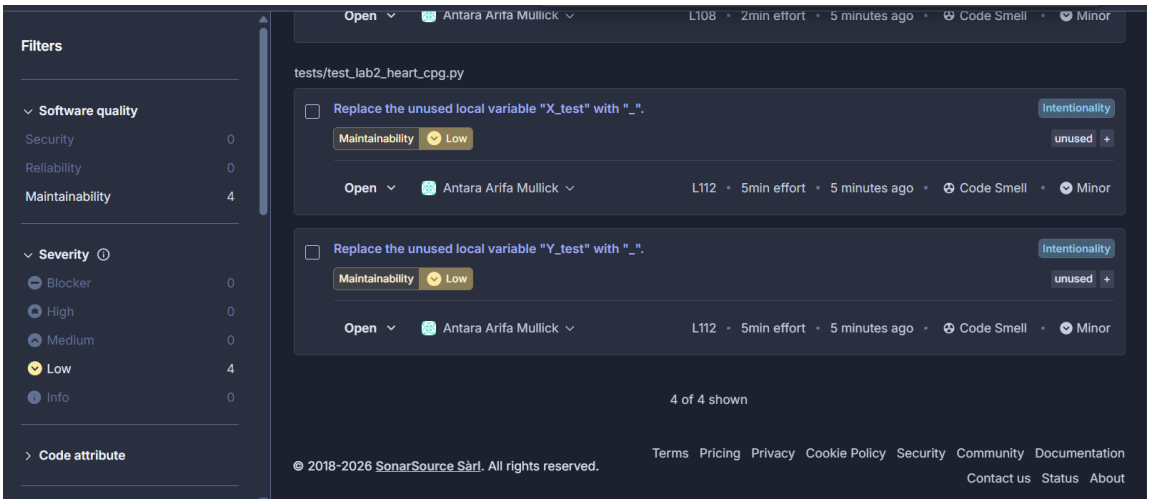


Figure 6: SonarCloud maintainability issues identifying unused local variables in the test code.

The Measures view was also inspected to see the project-level quality categories available for further analysis, including security, reliability, maintainability, duplication, size, complexity, and coverage-related measures.



Figure 7: SonarCloud Measures view used to inspect available project-level quality metrics.

Therefore, Lab 1 primarily supported **Program Comprehension**, while the metrics and runtime information also provided supporting evidence for later impact and improvement decisions.

3.2 Structural Reverse Engineering and Impact Analysis – Lab 2

Lab 2 was used for structural analysis and reverse engineering through a **Code Property Graph (CPG)**. A CPG was generated from the project source using `pysrc2cpg` and imported into Joern. Joern queries were then used to inspect methods, control structures, literals, calls, and other source-code elements.

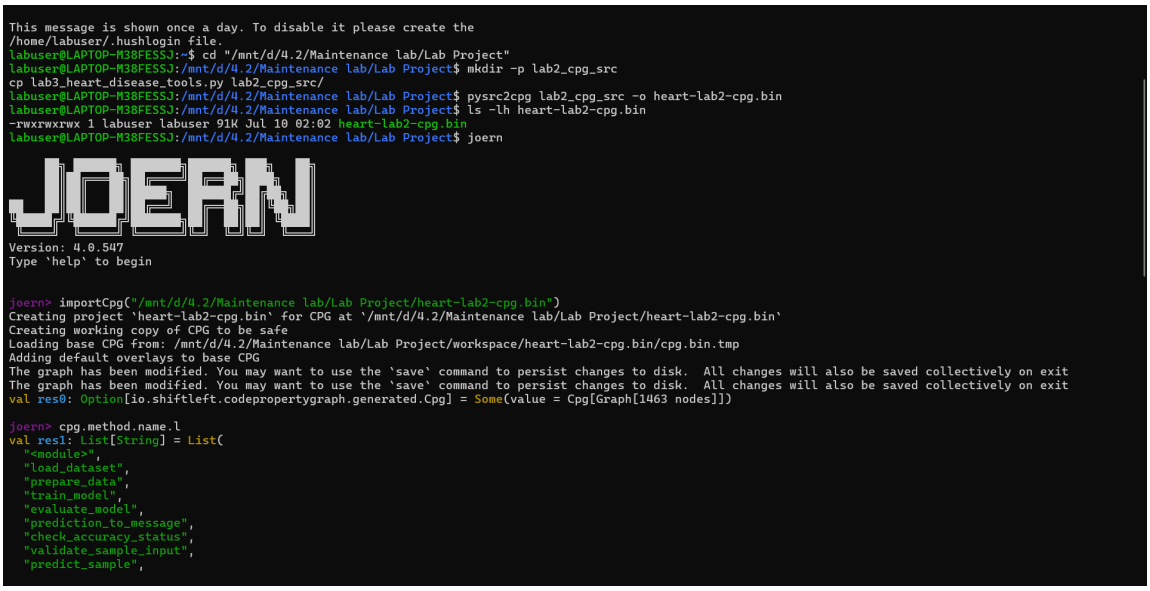


Figure 8: Generation and import of the Heart Disease Prediction CPG in Joern, followed by method inspection.

The CPG made the internal structure easier to inspect because source elements and their relationships could be queried instead of reading the complete source manually. For example, control structures and their line locations were retrieved directly from the graph representation.

```
joern> cpg.file.name.l
val res2: List[String] = List("lab3_heart_disease_tools.py", "N/A")

joern> cpg.controlStructure.code.l
val res3: List[String] = List(
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "if ... : ...",
  "while ... : ...",
  "if ... : ...",
  "if ... : ..."
)

joern> cpg.controlStructure.map(c => (c.lineNumber, c.code)).l
val res4: List[(Option[Int], String)] = List(
  (Some(value = 127), "if ... : ..."),
  (Some(value = 129), "if ... : ..."),
  (Some(value = 134), "if ... : ..."),
  (Some(value = 137), "if ... : ..."),
  (Some(value = 148), "if ... : ..."),
  (Some(value = 153), "if ... : ..."),
  (Some(value = 156), "if ... : ..."),
  (Some(value = 168), "if ... : ..."),
  (Some(value = 171), "if ... : ..."),
  (Some(value = 175), "if ... : ..."),
  (Some(value = 174), "while ... : ..."),
  (Some(value = 195), "if ... : ..."),
  (Some(value = 241), "if ... : ...")
)

joern> cpg.literal.code.l
val res5: List[String] = List(
  """""""
```

Figure 9: Joern queries used to inspect control structures and other CPG elements.

The graph was also exported for Neo4j analysis. The exported graph contained **1067 nodes** and **2083 relationships**. These relationships provided a structural basis for understanding dependencies and considering which parts might be affected by a change.

```
Created 14 relationships
0 rows
ready to start consuming query after 165 ms, results consumed after another
0 ms
Created 81 relationships
0 rows
ready to start consuming query after 350 ms, results consumed after another
0 ms
Created 335 relationships
0 rows
ready to start consuming query after 83 ms, results consumed after another 1
 ms
Created 13 relationships
labuser@LAPTOP-M38FESSJ:/mnt/d/4.2/Maintenance lab/Lab Project$ ~/maintenanc
e_lab/neo4j/bin/cypher-shell -u neo4j -p labpass123
Connected to Neo4j using Bolt protocol version 5.7 at neo4j://localhost:7687
 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> MATCH (n) RETURN count(n);
+-----------+
| count(n) |
+-----------+
| 1067     |
+-----------+

1 row
ready to start consuming query after 35 ms, results consumed after another 1
 ms
neo4j@neo4j> MATCH ()-[r]->() RETURN count(r);
+-----------+
| count(r) |
+-----------+
| 2083     |
+-----------+

1 row
ready to start consuming query after 40 ms, results consumed after another 1
 ms
```

Figure 10: Neo4j queries confirming 1067 nodes and 2083 relationships in the exported graph.

Neo4j graph visualization provided a visual form of reverse engineering, showing connected program entities and their relationships. A method-level query first displayed the methods represented in the graph and their AST connections from the module node.

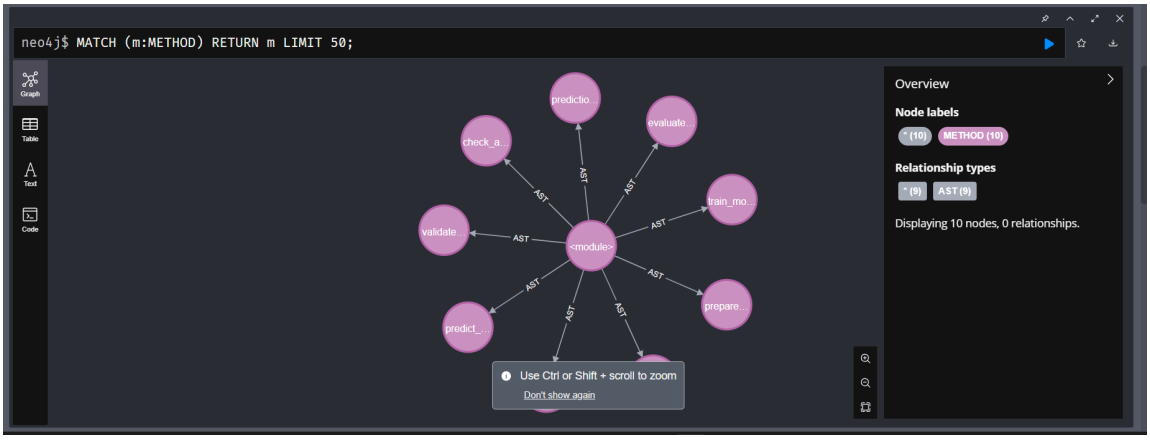


Figure 11: Neo4j method-level view showing methods and AST relationships from the module representation.

Individual maintenance-sensitive methods were then expanded to inspect their internal AST neighborhoods. The `prediction_to_message` view exposed its blocks, control structures, calls, identifiers, literals, returns, and related nodes.

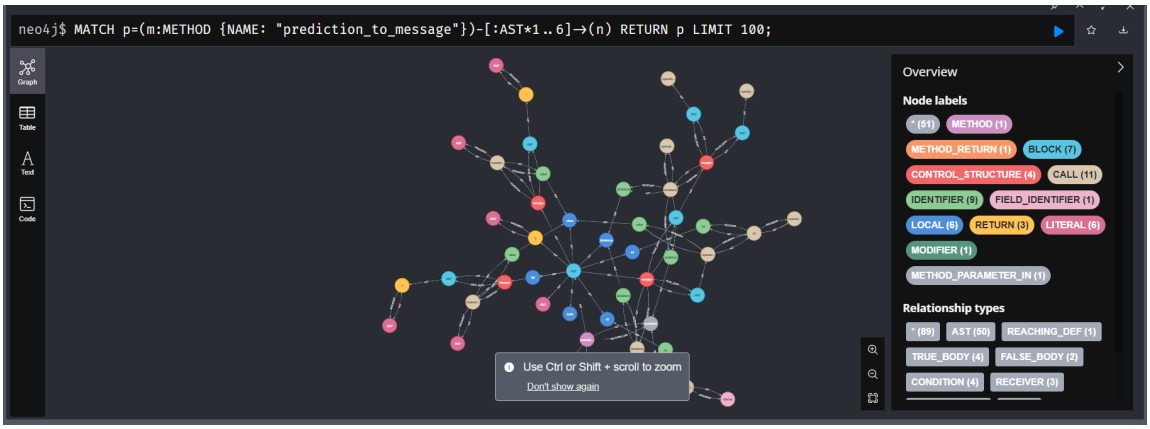


Figure 12: Neo4j AST visualization for the `prediction_to_message` method.

The same type of query was applied to `check_accuracy_status`, making its decision-oriented internal structure visible as a graph.

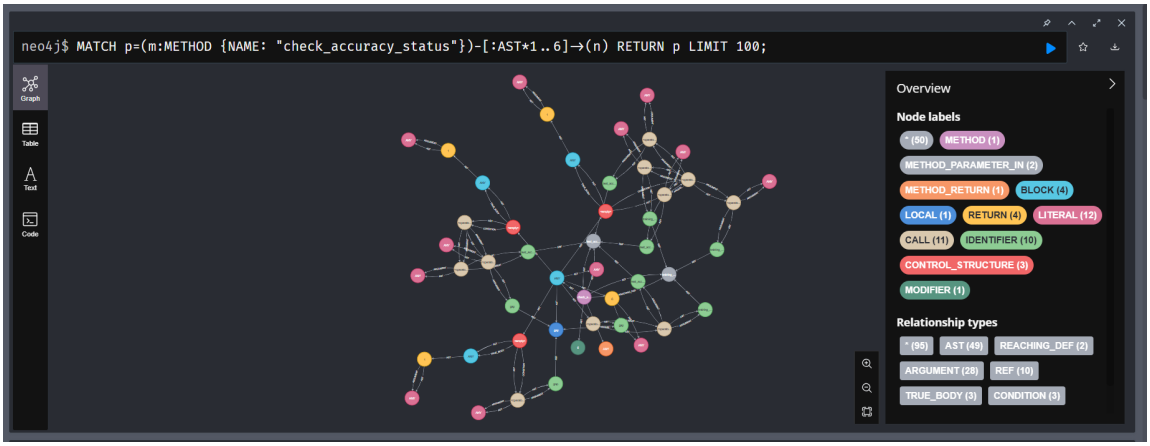


Figure 13: Neo4j AST visualization for the `check_accuracy_status` method.

The sample-input validation function was also inspected to expose its control structures, calls, identifiers, local values, and return paths.

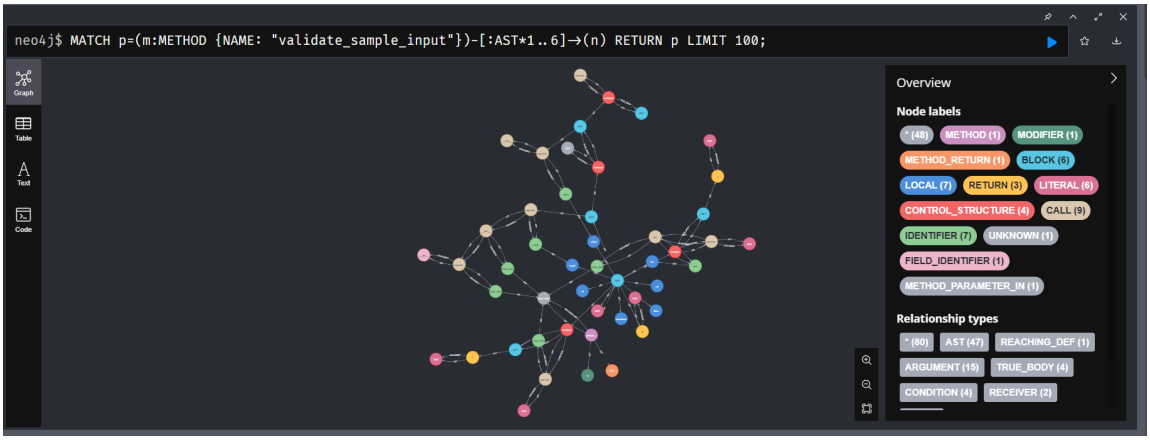


Figure 14: Neo4j AST visualization for the `validate_sample_input` method.

Finally, a query over `CONTROL_STRUCTURE` nodes and their `CONDITION` relationships provided a focused view of branch conditions across the analyzed source.

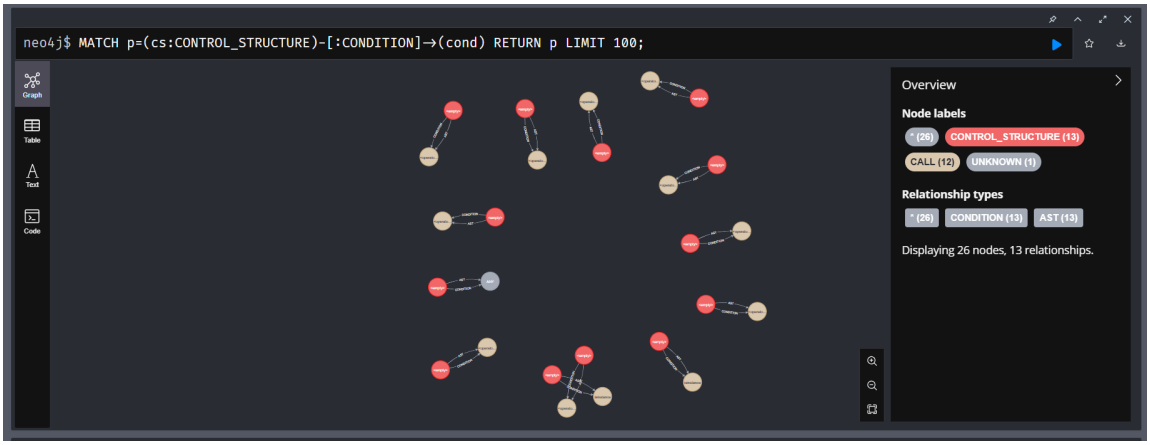


Figure 15: Neo4j query visualizing control-structure nodes and their condition relationships.

Thus, Lab 2 directly supported **Reverse Engineering**, **Program Comprehension**, and **Impact Analysis** by exposing the structural relationships within the existing system.

3.3 Runtime Comprehension and Performance Analysis – Lab 3

Lab 3 applied four transformation and profiling tools to the same Heart Disease Prediction project: **Loguru**, **PySnooper**, **VizTracer**, and **SnakeViz**. These tools provided different runtime views that are useful during software maintenance.

Loguru recorded dataset loading, dataset shape, missing-value information, model training, accuracy, and final prediction. This provided a persistent execution history that can support fault localization and maintenance diagnosis.

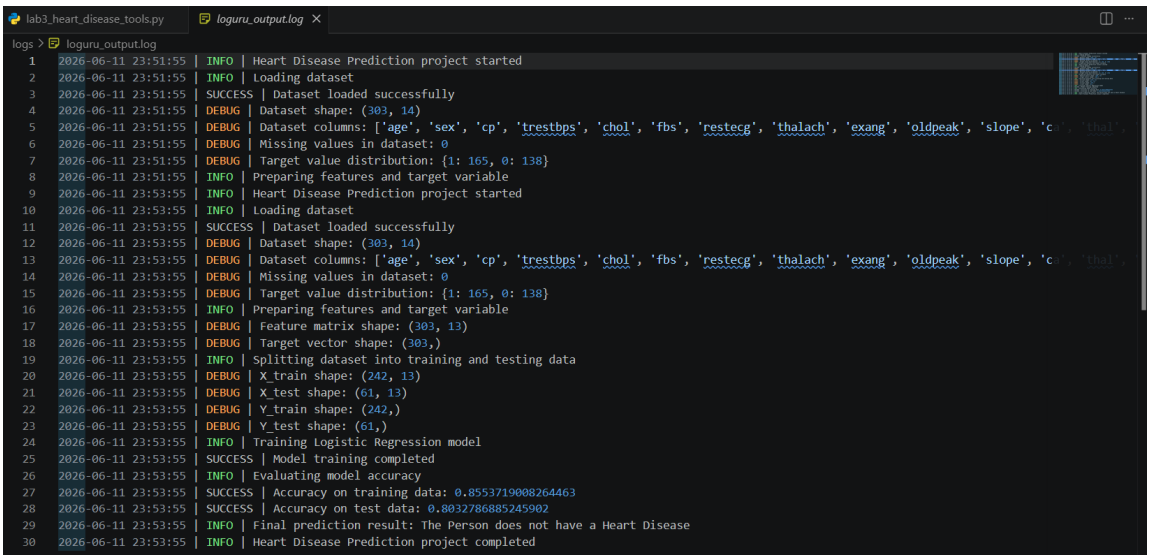


Figure 16: Loguru output showing structured runtime information for the project.

PySnooper was applied to the prediction function. It showed the function line by line, including the input tuple, NumPy conversion, reshaping, prediction, and final message. This was useful for detailed program comprehension of a maintenance-sensitive function.

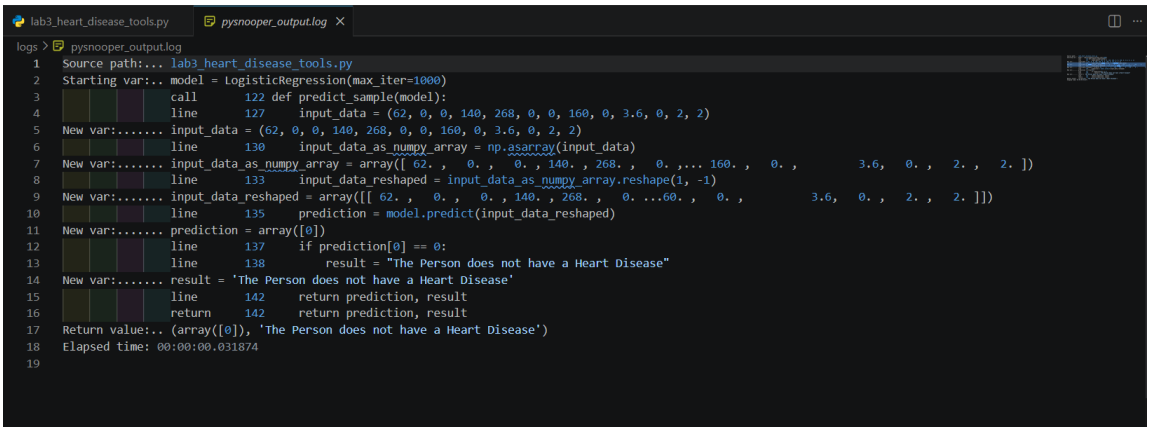


Figure 17: PySnooper line-by-line trace of the prediction workflow.

VizTracer generated an HTML execution timeline that represented the runtime sequence and function activity. It provided a dynamic reverse-engineering view of how the application behaved over time.

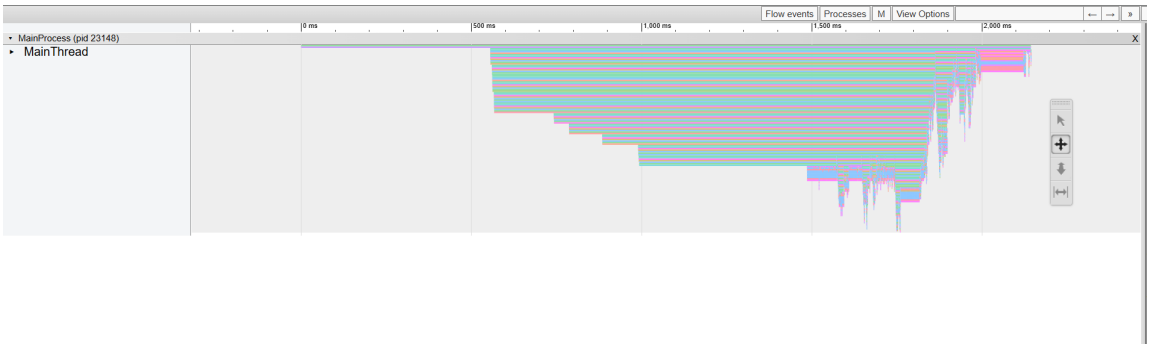


Figure 18: VizTracer execution timeline of the Heart Disease Prediction project.

SnakeViz visualized cProfile data as an icicle chart. It showed where execution time was distributed across project and library calls, providing information useful for perfective maintenance and performance investigation.

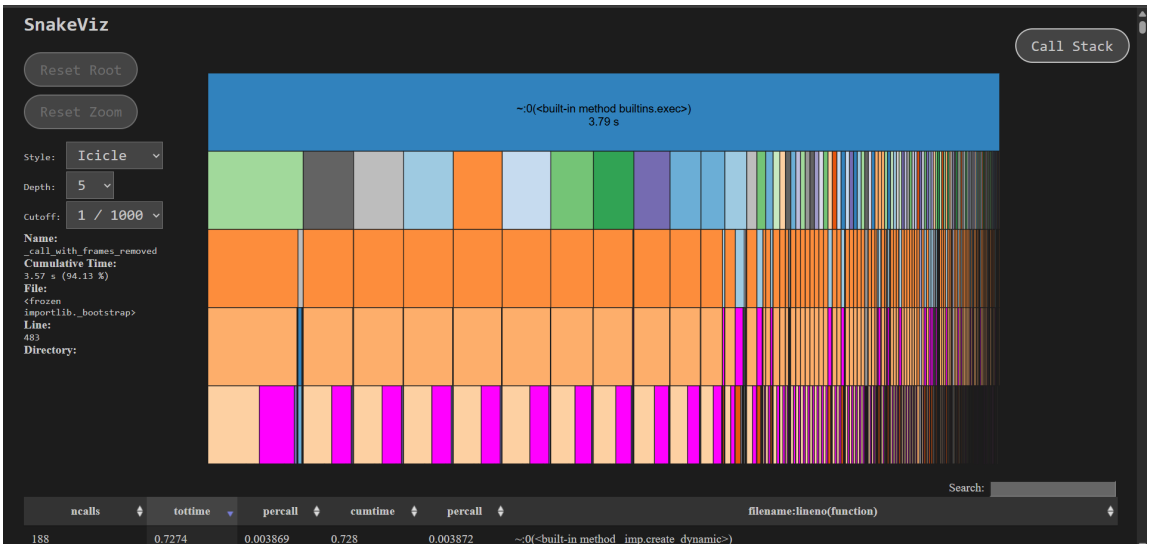


Figure 19: SnakeViz profiling result showing execution-time distribution.

Therefore, Lab 3 mainly supported **Runtime Program Comprehension**, dynamic reverse engineering, debugging support, and performance-oriented analysis.

3.4 Clone Detection and Refactoring Opportunity Analysis – Lab 5

Lab 5 applied **jscpd** clone detection to the integrated project. The analysis examined **5 files** containing approximately **830 lines of code**. It detected **1 clone** containing **241 duplicated lines**, corresponding to approximately **29.04%** duplication in the analyzed set.

```
D:\4.2\Maintenance lab\Lab5>npx --yes jscpd@5 . --format python --min-lines 5 --min-tokens 50 --reporters console,html,j
son --output jscpd-report-default --ignore "**/.venv/**,**/venv/**,**/__pycache__/**,**/outputs/**,**/logs/**,**/htmlcov
/**,**/jscpd-report-default/**,**/jscpd-report-sensitive/**"
Clone found (python)
 - lab2_cpg_src\lab3_heart_disease_tools.py [1:1 - 242:11] (242 lines, 973 tokens)
   lab3_heart_disease_tools.py [1:1 - 242:11]
```

Format	Files analyzed	Total lines	Total tokens	Clones found	Duplicated lines	Duplicated tokens
python	5	830	3481	1	241 (29.04%)	973 (27.95%)
Total:	5	830	3481	1	241 (29.04%)	973 (27.95%)

```
Found 1 clones.
HTML report saved to jscpd-report-default\jscpd-report.html
JSON report saved to jscpd-report-default\jscpd-report.json
time: 213.184ms

💡 Auto-refactor with AI: npx skills add https://github.com/kucherenko/jscpd --skill dry-refactoring
🚨 New: Gangsta Agents – discipline your AI coding → gangsta.page
💖 Support jscpd project → https://opencollective.com/jscpd

D:\4.2\Maintenance lab\Lab5>
```

Figure 20: jscpd execution and terminal summary for clone detection.

The generated HTML report summarized the same clone result. The detected duplication was mainly caused by the copy of `lab3_heart_disease_tools.py` that had been placed inside `lab2_cpg_src` for the Lab 2 CPG analysis. Therefore, the result correctly demonstrated clone detection while also identifying a real maintenance concern: duplicated source copies can become inconsistent if one copy changes and the other does not.

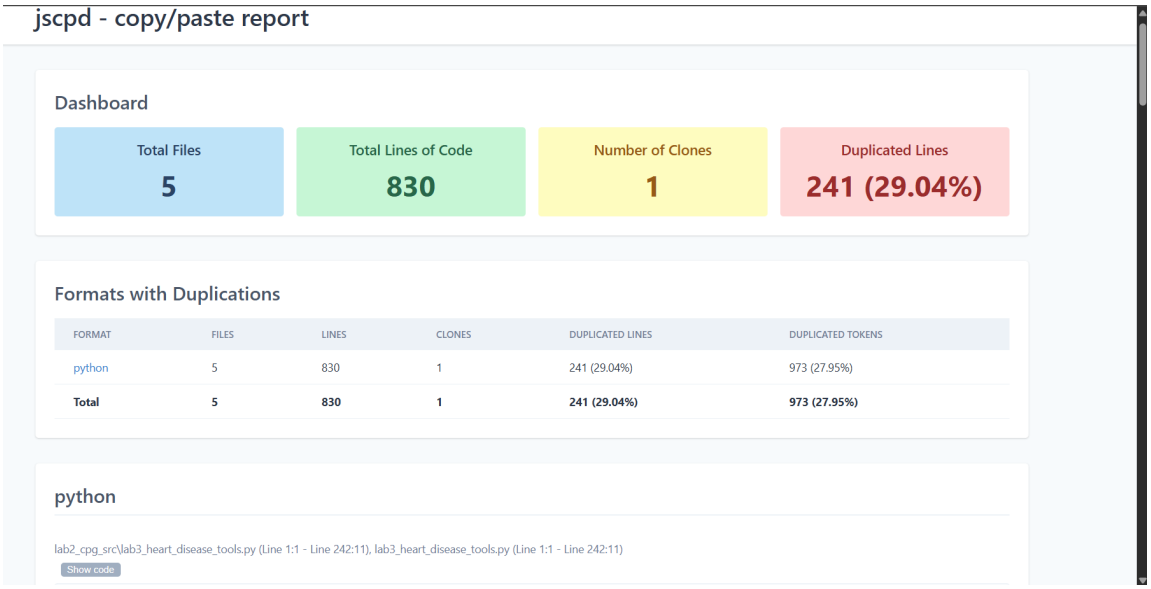


Figure 21: jscpd HTML report showing one detected clone and 29.04% duplicated lines.

Lab 5 therefore supported **Preventive Maintenance analysis**, **Impact Analysis**, and identification of **Refactoring Opportunities**. The lab detected the opportunity; the later maintenance activities performed actual refactoring changes.

4 Corrective Maintenance

4.1 Problem Identified

Corrective maintenance was performed on sample-input validation. The existing `validate_sample_input` function checked whether each value was numeric. However, `NaN` and infinity are floating-point values, so they passed the numeric-type test and were incorrectly reported as valid input.

The defect was reproduced before changing the code. Both `np.nan` and `np.inf` returned `valid-input`.

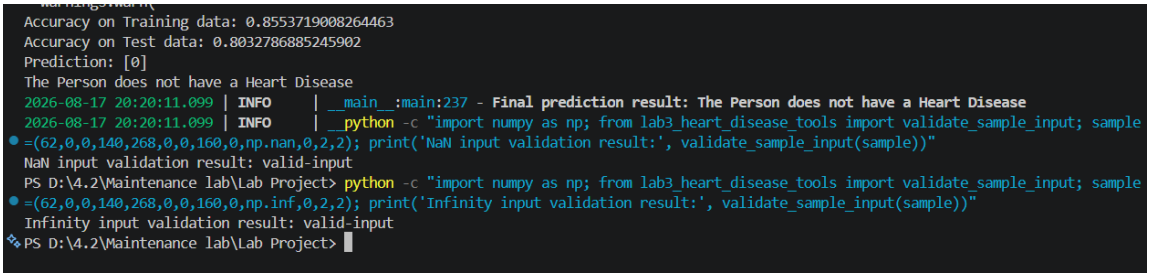


Figure 22: Before corrective maintenance: NaN and infinity were incorrectly accepted as valid input.

4.2 Program Comprehension

The prediction workflow was studied as a focused continuation of the earlier static, dynamic, CPG, and runtime analysis. The relevant flow was `main()` → `predict_sample()` → `validate_sample_input()` before the model prediction was generated. This showed that correcting validation at this point could prevent invalid numeric values from reaching the prediction operation.

4.3 Change Management

The change was managed as a small documented corrective request:

Item	Corrective Change
Problem	NaN and infinity were accepted as valid patient input.
Requested change	Reject non-finite numerical values before prediction.
Affected function	`validate_sample_input()`
Implementation	Added a finite-number check using `np.isfinite()`.
Expected behavior	NaN and infinity return `non-finite-input`.

4.4 Impact Analysis

A project-wide source search was performed for `validate_sample_input` and `predict_sample`. The search showed that the validation function was called from the prediction workflow and was also referenced by the test file. The change was therefore localized to sample validation and prediction behavior. Dataset loading, preparation, training, and accuracy evaluation did not require modification.

```
PS D:\4.2\Maintenance lab\Lab Project> Get-ChildItem -Path . -Recurse -Filter *.py | Select-String -Pattern "validate_sample_input|predi
> ct_sample"

lab3_heart_disease_tools.py:162:def validate_sample_input(input_data):
lab3_heart_disease_tools.py:204:def predict_sample(model):
lab3_heart_disease_tools.py:211:    input_status = validate_sample_input(input_data)
lab3_heart_disease_tools.py:249:    prediction, result = predict_sample(model)
lab2_cpg_src\lab3_heart_disease_tools.py:162:def validate_sample_input(input_data):
lab2_cpg_src\lab3_heart_disease_tools.py:185:def predict_sample(model):
lab2_cpg_src\lab3_heart_disease_tools.py:192:    input_status = validate_sample_input(input_data)
lab2_cpg_src\lab3_heart_disease_tools.py:230:    prediction, result = predict_sample(model)
tests\test_lab2_heart_cpg.py:8:    validate_sample_input,
tests\test_lab2_heart_cpg.py:13:    predict_sample,
tests\test_lab2_heart_cpg.py:27:    assert validate_sample_input(sample) == "valid-input"
tests\test_lab2_heart_cpg.py:63:    assert validate_sample_input(None) == "missing-input"
tests\test_lab2_heart_cpg.py:68:    assert validate_sample_input(sample) == "invalid-input-length"
tests\test_lab2_heart_cpg.py:73:    assert validate_sample_input(sample) == "non-numeric-input"
tests\test_lab2_heart_cpg.py:110:def test_predict_sample():
tests\test_lab2_heart_cpg.py:115:    prediction, result = predict_sample(model)
```

Figure 23: Reference search used for corrective-maintenance impact analysis.

4.5 Reverse Engineering

A focused AST-based call analysis reconstructed the relationships around the changed area. It confirmed that `predict_sample()` calls both `validate_sample_input()` and

`prediction_to_message()`, while `main()` calls `predict_sample()`. This source-level reverse engineering supported the earlier impact analysis.

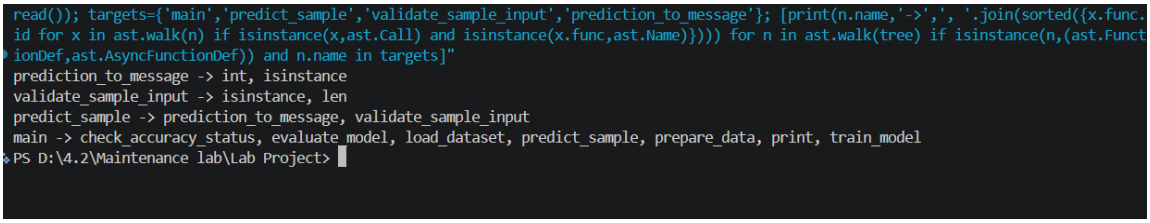


Figure 24: AST-based reverse engineering of the corrective-maintenance call relationships.

4.6 Refactoring

During the corrective work, `prediction_to_message()` was also refactored without changing its intended behavior. Repeated handling of NumPy arrays, lists, and tuples was combined, and the separate prediction-value conditions were replaced with a small message mapping. This reduced repeated logic and made the conversion function easier to extend and read.

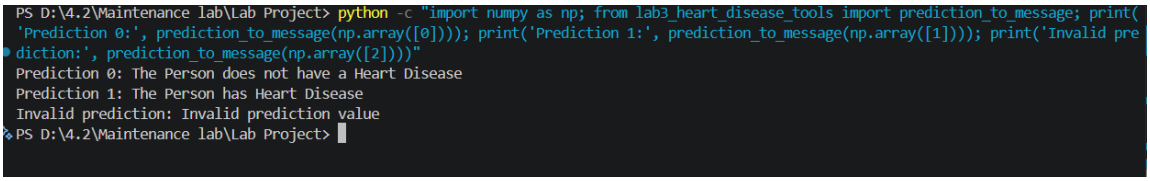


Figure 25: Verification of the refactored prediction-to-message behavior for values 0, 1, and an invalid value.

4.7 Result

After adding the finite-number check, both NaN and infinity were correctly rejected as `non-finite-input`. The before-and-after outputs directly demonstrate the corrective change.

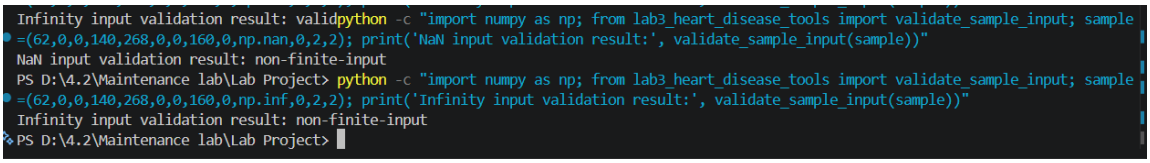


Figure 26: After corrective maintenance: NaN and infinity are rejected as non-finite input.

Corrective maintenance was therefore completed by identifying an existing defect, understanding and analyzing its impact, modifying the validation logic, refactoring related code, and verifying the corrected behavior.

5 Adaptive Maintenance

5.1 Maintenance Requirement

The application originally used relative paths such as `heart_disease_data.csv` and paths under `logs/`. This meant that the program depended on the current working directory. When the script was launched from the parent `Maintenance lab` directory, it could not find the dataset and raised `FileNotFoundError`.

```
PS D:\4.2\Maintenance lab\Lab Project> cd ..
PS D:\4.2\Maintenance lab> python ".\Lab Project\lab3_heart_disease_tools.py"
2026-08-17 21:13:55.323 | INFO     | __main__:main:228 - Heart Disease Prediction project started
2026-08-17 21:13:55.323 | INFO     | __main__:load_dataset:51 - Loading dataset
Traceback (most recent call last):
  File "D:\4.2\Maintenance lab\Lab Project\lab3_heart_disease_tools.py", line 257, in <module>
    main()
  File "D:\4.2\Maintenance lab\Lab Project\lab3_heart_disease_tools.py", line 230, in main
    heart_data = load_dataset("heart_disease_data.csv")
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "D:\4.2\Maintenance lab\Lab Project\lab3_heart_disease_tools.py", line 52, in load_dataset
    heart_data = pd.read_csv(file_path)
                 ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\USER\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py", lin
e 1026, in read_csv
    return _read(filepath_or_buffer, kwds)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\USER\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py", lin
e 620, in _read
    parser = TextFileReader(filepath_or_buffer, **kwds)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\USER\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py", lin
e 1620, in __init__
    self._engine = self._make_engine(f, self.engine)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\USER\AppData\Roaming\Python\Python312\site-packages\pandas\io\parsers\readers.py", lin
e 1880, in _make_engine
    self.handles = get_handle(
                   ^^^^^^^^^^^
  File "C:\Users\USER\AppData\Roaming\Python\Python312\site-packages\pandas\io\common.py", line 873, in
 get_handle
    handle = open(
             ^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'heart_disease_data.csv'
PS D:\4.2\Maintenance lab>
```

Figure 27: Before adaptive maintenance: running the project from another working directory caused FileNotFoundError.

5.2 Program Comprehension

The path-handling code was inspected to determine which resources depended on the execution environment. The source contained relative references for the dataset, Loguru output, PySnooper output, and profiling/output locations.

5.3 Change Management

The adaptive change was managed around a portability requirement:

Item	Adaptive Change
Existing problem	Resource paths depended on the current terminal directory.
Environmental change	The program may be launched from a different directory.
Requested adaptation	Resolve project resources relative to the script location.
Implementation	Centralized paths using `pathlib.Path`.
Expected behavior	The application runs correctly from inside or outside the project folder.

5.4 Impact Analysis

A source search was performed for `heart_disease_data.csv`, `logs/`, and `outputs/`. It identified the path-related locations in the main source file, including dataset loading, Loguru, PySnooper, and profiling references. This showed that path handling could be changed centrally without changing the machine-learning algorithm.

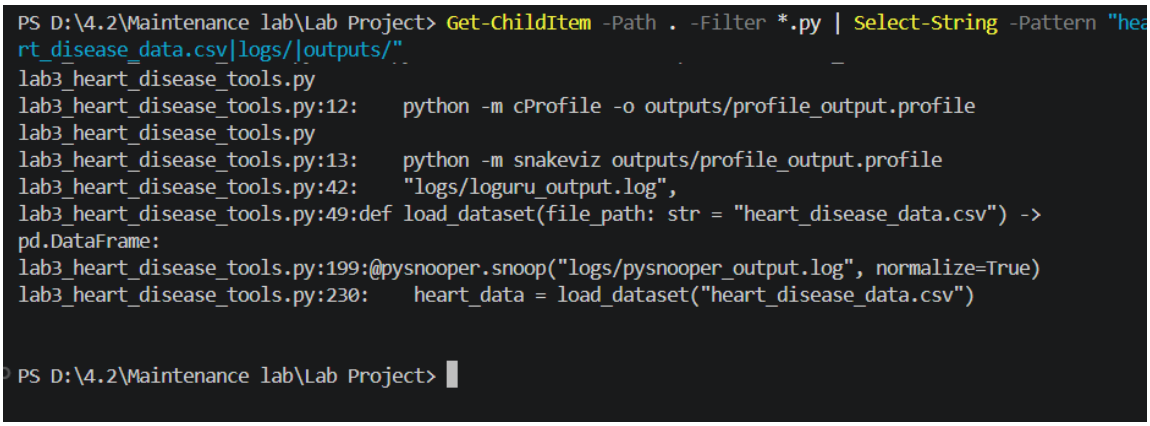


Figure 28: Search results used for program comprehension and adaptive impact analysis of path-dependent resources.

5.5 Reverse Engineering

Focused AST analysis showed that `main()` calls `load_dataset()`, and `load_dataset()` reaches `read_csv`. This reconstructed the relevant resource dependency and confirmed the correct location for adapting dataset access.

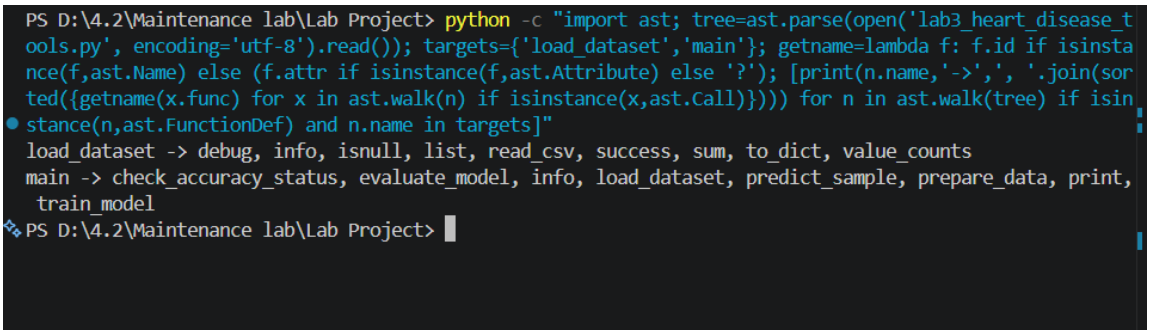


Figure 29: AST-based reverse engineering of the dataset-loading dependency.

5.6 Refactoring

Scattered relative paths were replaced by centralized path constants based on `BASE_DIR = Path(__file__).resolve().parent`. Named paths were created for the dataset, logs, and outputs. The dataset-loading function was also simplified so that the main workflow could call it without passing a hard-coded relative file name. This refactoring improved readability as well as portability.

5.7 Result

After the adaptive change, the exact scenario that had previously failed was repeated from the parent directory. The dataset loaded successfully, the model trained and evaluated, and the final prediction was produced normally.

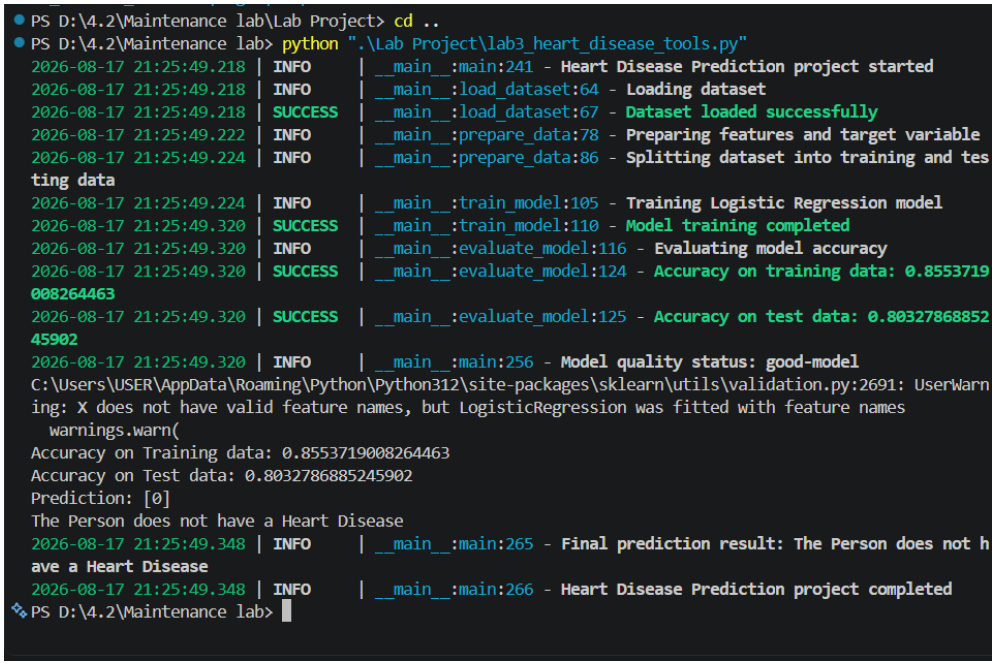


Figure 30: After adaptive maintenance: the application runs successfully from outside the project directory.

The project was therefore adapted to a changed execution environment without altering its core prediction behavior.

6 Preventive Maintenance

6.1 Maintenance Risk

The original loader assumed that any readable CSV file contained the correct heart-disease schema. To demonstrate the maintenance risk, a temporary dataset was created without the required `target` column. Pandas could still read the file, so the program logged that the dataset had loaded, but the later attempt to access `target` caused an uncontrolled `KeyError`.

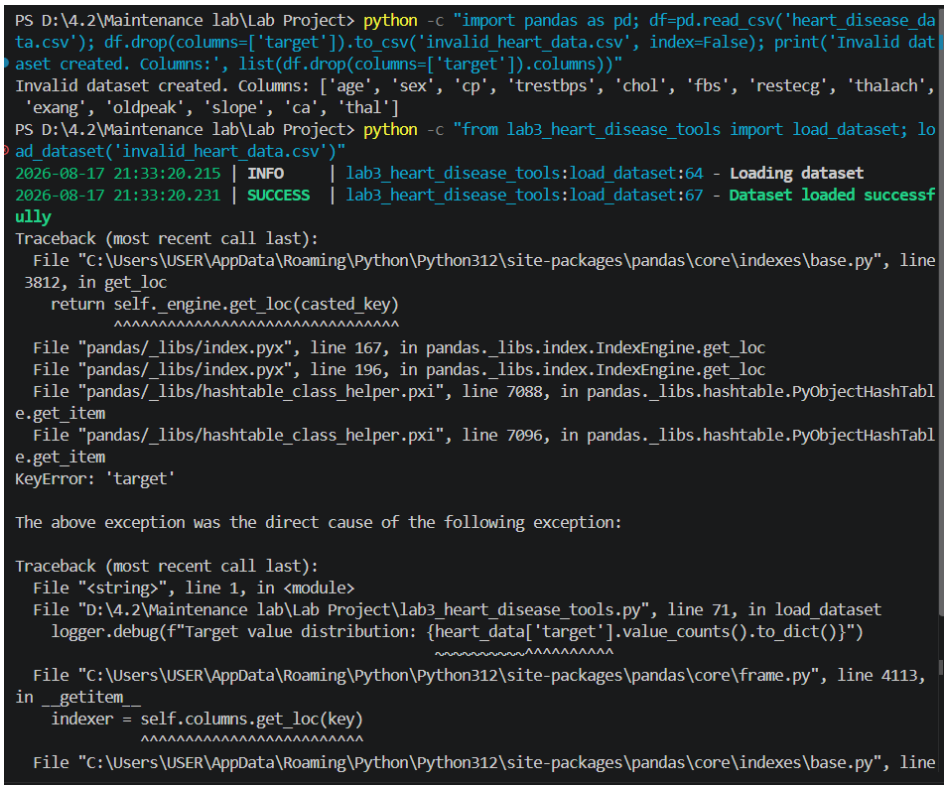


Figure 31: Before preventive maintenance: a structurally invalid dataset caused KeyError for the missing target column.

This was a preventive-maintenance scenario because the normal dataset was currently valid; the purpose of the change was to protect the system from future dataset changes or corruption.

6.2 Program Comprehension

The existing data workflow was inspected to determine where assumptions about the dataset schema were made. The main dependencies were in `load_dataset()` and `prepare_data()`, both of which directly used the `target` column.

6.3 Change Management

Item	Preventive Change
Current risk	The application assumed the dataset schema was always valid.
Possible future failure	Missing columns, missing values, or invalid target values could break processing later.
Requested change	Validate the dataset immediately after reading it.
Implementation	Added `validate_dataset_schema()` and centralized schema constants.
Expected behavior	Invalid datasets are rejected early with clear validation errors.

6.4 Impact Analysis

A search for `target`, `load_dataset`, and `prepare_data` identified all important schema-dependent locations. This confirmed that the preventive change should be placed between CSV loading and feature/target preparation.

```
PS D:\4.2\Maintenance lab\Lab Project> Get-ChildItem -Path . -Filter *.py | Select-String -Pattern "tar
get|load_dataset|prepare_data"

lab1_dynamic_trace.py:15:TARGET_FILE = os.path.abspath("lab3_heart_disease_tools.py")
lab1_dynamic_trace.py:33:    if current_file != TARGET_FILE:
lab1_dynamic_trace.py:84:        file.write(f"Target file: {TARGET_FILE}\n")
lab3_heart_disease_tools.py:62:def load_dataset(file_path=DATASET_PATH) -> pd.DataFrame:
lab3_heart_disease_tools.py:71:    logger.debug(f"Target value distribution:
{heart_data['target'].value_counts().to_dict()}")
lab3_heart_disease_tools.py:76:def prepare_data(heart_data: pd.DataFrame):
lab3_heart_disease_tools.py:77:    """Separate features/target and split the dataset."""
lab3_heart_disease_tools.py:78:    logger.info("Preparing features and target variable")
lab3_heart_disease_tools.py:80:    X = heart_data.drop(columns=["target"])
lab3_heart_disease_tools.py:81:    Y = heart_data["target"]
lab3_heart_disease_tools.py:84:    logger.debug(f"Target vector shape: {Y.shape}")
lab3_heart_disease_tools.py:243:    heart_data = load_dataset()
lab3_heart_disease_tools.py:244:    X_train, X_test, Y_train, Y_test = prepare_data(heart_data)


PS D:\4.2\Maintenance lab\Lab Project>
```

Figure 32: Schema-dependent source locations identified during preventive impact analysis.

6.5 Reverse Engineering

AST-based call analysis reconstructed the updated flow. The result showed that `load_dataset()` now calls `validate_dataset_schema()`, while `main()` continues through `load_dataset()` and `prepare_data()`. The validation therefore occurs before model preparation and training.

```
PS D:\4.2\Maintenance lab\Lab Project> python -c "import ast; tree=ast.parse(open('lab3_heart_disease_t
ools.py', encoding='utf-8').read()); targets={'load_dataset','validate_dataset_schema','prepare_data','
main'}; getname=lambda f: f.id if isinstance(f,ast.Name) else (f.attr if isinstance(f,ast.Attribute) el
se '?'); [print(n.name,'->',', '.join(sorted({getname(x.func) for x in ast.walk(n) if isinstance(x,ast.
Call)}))) for n in ast.walk(tree) if isinstance(n,ast.FunctionDef) and n.name in targets]"
validate_dataset_schema -> ValueError, any, isnull, issubset, list, set, unique
load_dataset -> debug, info, isnull, list, read_csv, success, sum, to_dict, validate_dataset_schema, va
lue_counts
prepare_data -> debug, drop, info, train_test_split
main -> check_accuracy_status, evaluate_model, info, load_dataset, predict_sample, prepare_data, print,
 train_model
PS D:\4.2\Maintenance lab\Lab Project>
```

Figure 33: Reverse-engineered data-loading and schema-validation call relationships.

6.6 Refactoring

The repeated literal string `"target"` was replaced with the named constant `TARGET_COLUMN`. The expected feature names were also centralized in `FEATURE_COLUMNS`, with `REQUIRED_COLUMNS` representing the full schema. This removed scattered schema assumptions and made future dataset-related changes easier to manage.

6.7 Result

After the change, the invalid dataset was rejected immediately with a controlled and descriptive error identifying the missing column instead of failing later with a raw `KeyError`.

```
PS D:\4.2\Maintenance lab\Lab Project> python -c "from lab3_heart_disease_tools import load_dataset; lo
ad_dataset('invalid_heart_data.csv')"
2026-08-17 22:18:28.546 | INFO     | lab3_heart_disease_tools:load_dataset:114 - Loading dataset
Traceback (most recent call last):
  File "<string>", line 1, in <module>
  File "D:\4.2\Maintenance lab\Lab Project\lab3_heart_disease_tools.py", line 117, in load_dataset
    validate_dataset_schema(heart_data)
  File "D:\4.2\Maintenance lab\Lab Project\lab3_heart_disease_tools.py", line 92, in validate_dataset_s
chema
    raise ValueError(
ValueError: Dataset validation failed. Missing required columns: ['target']
PS D:\4.2\Maintenance lab\Lab Project>
```

Figure 34: After preventive maintenance: the missing target column is detected by explicit dataset validation.

The final project execution used the valid dataset successfully, confirming that the added validation protected the application without preventing normal operation.

7 Perfective Maintenance

7.1 Improvement Requirement

The working project originally contained a hard-coded patient sample inside `predict_sample()`. Predicting a different sample required opening the source file and editing the tuple manually.

The application also displayed only the predicted class and message.

A source search confirmed that the sample data was embedded inside the prediction function and that `main()` called `predict_sample(model)` without any user-provided sample.

```
PS D:\4.2\Maintenance lab\Lab Project> Get-ChildItem -Path . -Filter *.py | Select-String -Pattern "def
 predict_sample|input_data =|model.predict|prediction_to_message|predict_sample\(model\)"

lab3_heart_disease_tools.py:173:    X_train_prediction = model.predict(X_train)
lab3_heart_disease_tools.py:176:    X_test_prediction = model.predict(X_test)
lab3_heart_disease_tools.py:189:def prediction_to_message(prediction):
lab3_heart_disease_tools.py:191:    Convert model prediction output into a readable message.
lab3_heart_disease_tools.py:192:    The model prediction is usually 0 or 1.
lab3_heart_disease_tools.py:268:def predict_sample(model):
lab3_heart_disease_tools.py:273:    input_data = (62, 0, 0, 140, 268, 0, 0, 160, 0, 3.6, 0, 2, 2)
lab3_heart_disease_tools.py:287:    prediction = model.predict(input_data_reshaped)
lab3_heart_disease_tools.py:289:    result = prediction_to_message(prediction)
lab3_heart_disease_tools.py:313:    prediction, result = predict_sample(model)


PS D:\4.2\Maintenance lab\Lab Project>
```

Figure 35: Before perfective maintenance: the prediction sample was hard-coded inside the source file.

7.2 Program Comprehension

The prediction workflow was examined from command execution through `main()`, `predict_sample()`, model prediction, and `prediction_to_message()`. This established where a user-facing improvement could be introduced without changing dataset preparation or model training.

7.3 Change Management

Item	Perfective Change
Existing limitation	Patient values were hard-coded in the source.
Requested improvement	Allow a user to provide 13 values from the command line.
Additional improvement	Display the model's predicted probability for the heart-disease class.
Main affected area	Prediction and command-line input workflow.
Expected behavior	Different samples can be evaluated without editing the program.

7.4 Impact Analysis

The earlier source search showed that the main affected components were `predict_sample()` and `main()`, together with input validation and prediction-message conversion. Dataset

loading, preparation, training, and evaluation were not structurally changed by this improvement.

7.5 Reverse Engineering

Focused AST analysis of the final workflow showed that `parse_sample_input()` calls `validate_sample_input()`, and `predict_sample()` calls validation, `predict`, `predict_proba`, and `prediction_to_message()`. The `main()` function now calls both the sample parser and the prediction function when custom input is provided.

```
Model probability for heart disease clapython -c "import ast; tree=ast.parse(open('lab3_heart_disease_t
ools.py', encoding='utf-8').read()); targets={'parse_sample_input','predict_sample','prediction_to_mess
age','main'}; getname=lambda f: f.id if isinstance(f,ast.Name) else (f.attr if isinstance(f,ast.Attribu
te) else '?'); [print(n.name,'->',', '.join(sorted({getname(x.func) for x in ast.walk(n) if isinstance(
x,ast.Call)}))) for n in ast.walk(tree) if isinstance(n,ast.FunctionDef) and n.name in targets]"
prediction_to_message -> get, int, isinstance
parse_sample_input -> ValueError, float, split, strip, tuple, validate_sample_input
predict_sample -> array, asarray, predict, predict_proba, prediction_to_message, reshape, snoop, str, v
alidate_sample_input
main -> check_accuracy_status, evaluate_model, info, len, load_dataset, parse_sample_input, predict_sam
ple, prepare_data, print, train_model
PS D:\4.2\Maintenance lab\Lab Project>
```

Figure 36: AST-based reverse engineering of the improved custom-prediction workflow.

7.6 Refactoring

The original hard-coded tuple was extracted into `DEFAULT_SAMPLE`. A new `parse_sample_input()` helper was introduced to convert comma-separated terminal values and reuse the existing validation logic. The prediction function was generalized to accept an optional input sample. These changes separated responsibilities and made the prediction component more reusable.

7.7 Result

The final application successfully accepted a sample using the `-sample` command-line option. It produced the normal class prediction and readable message, and also displayed the model-predicted probability for the heart-disease class. For the demonstrated sample, the probability shown was **11.40%**. This value represents the model output for that class and is not a clinical diagnosis.

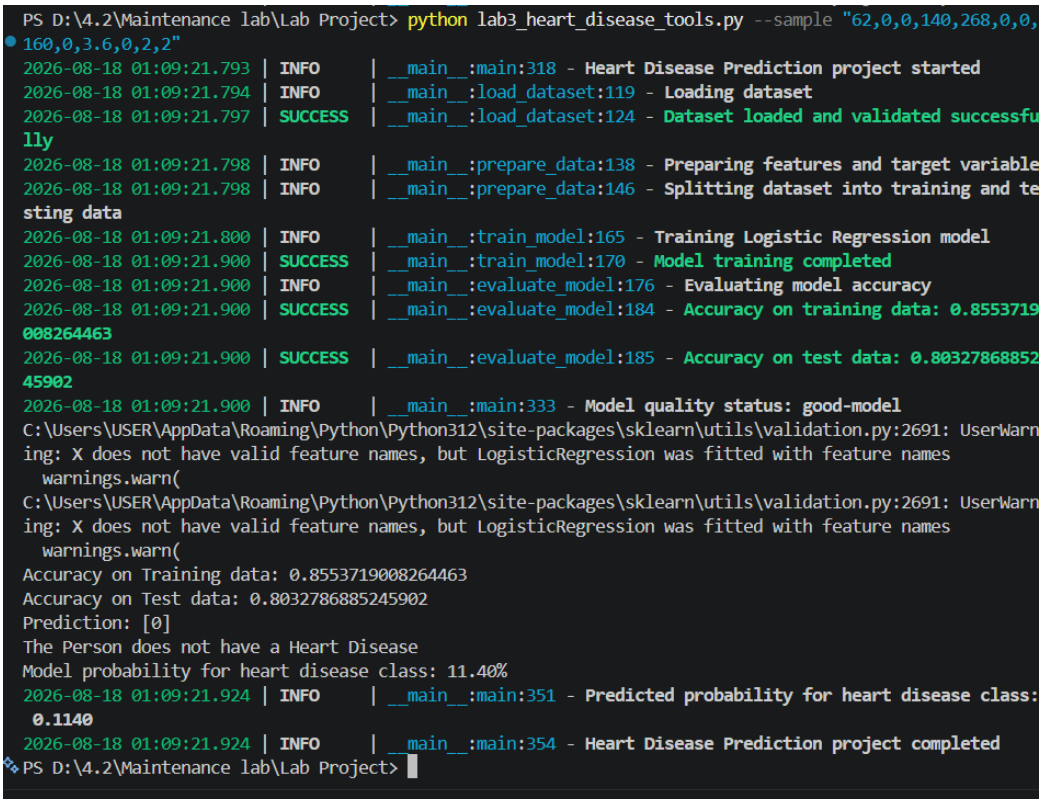


Figure 37: Perfective-maintenance result: custom command-line sample with prediction and model class probability.

The change improved usability and flexibility while preserving the existing model-training and evaluation workflow.

8 Conclusion

This project demonstrated that software maintenance is not only the act of changing code. Before a safe change can be made, the maintainer must understand the program, locate dependencies, determine the impact of the proposed modification, recover useful structural information, and improve the internal design where necessary.

The earlier lab activities created this understanding from several perspectives. Static and dynamic analysis described the source and execution behavior; the CPG and Neo4j exports exposed structural relationships; transformation tools showed runtime flow and performance; and clone detection identified duplicated source. These results then supported practical maintenance changes on the same project.

The four maintenance categories addressed different reasons for change. Corrective maintenance fixed an existing validation defect. Adaptive maintenance allowed the program to operate in a different execution environment. Preventive maintenance reduced the chance of future failure caused by invalid dataset structure. Perfective maintenance improved usability by allowing custom sample input and richer model output.